SIMATS ENGINEERING

ASSESSMENT TOOL 1 – SCENARIO BASED ASSIGNMENT

Question 1 – Greedy Best-First Search

Map data used: road distances g (Gate–Library 120, Gate–Canteen 200, Library–Lab 150, Library–Hostel 300, Canteen–Hostel 100, Canteen–Admin 260, Lab–Auditorium 350, Hostel–Auditorium 180, Admin–Auditorium 120) and straight-line distance h to the Auditorium (Gate 400, Library 330, Canteen 240, Lab 280, Hostel 160, Admin 110, Auditorium 0).

a) Search strategy

The robot uses Greedy Best-First Search, an informed (heuristic) search.

- It always expands the neighbour with the lowest heuristic value h(n) – the straight-line distance to the goal – so the evaluation is f(n) = h(n).
- It ignores g(n), the distance already travelled. This is exactly what the scenario describes ("does not consider how far it has already travelled"), which is the defining property of greedy search.

b) Trace of decisions

Step	Location expanded	Candidates (unvisited neighbours) with straight-line h	Decision
1	Gate	Library = 330, Canteen = 240	Canteen (240 < 330)
2	Canteen	Hostel = 160, Admin = 110 (Gate already visited)	Admin (110 < 160)
3	Admin	Auditorium = 0	Auditorium (goal reached)

c) Route, distance and optimality

Route: Gate → Canteen → Admin → Auditorium Distance = 200 + 260 + 120 = 580 m

Not guaranteed to be shortest. Greedy search is neither optimal nor complete in general because it looks only at the estimate, never at the cost already paid. Comparison of all routes:

Route	Total distance (m)	Remark
Gate → Canteen → Hostel → Auditorium (200+100+180)	480	Shortest
Gate → Canteen → Admin → Auditorium (200+260+120)	580	Greedy route
Gate → Library → Hostel → Auditorium (120+300+180)	600	
Gate → Library → Lab → Auditorium (120+150+350)	620	

The greedy route is 100 m (about 21%) longer than the best route. The mistake happens at the Canteen: Admin looks closer (110 vs 160) but the road to it costs 260 m, whereas Hostel is only 100 m away. Greedy search cannot see this cost.

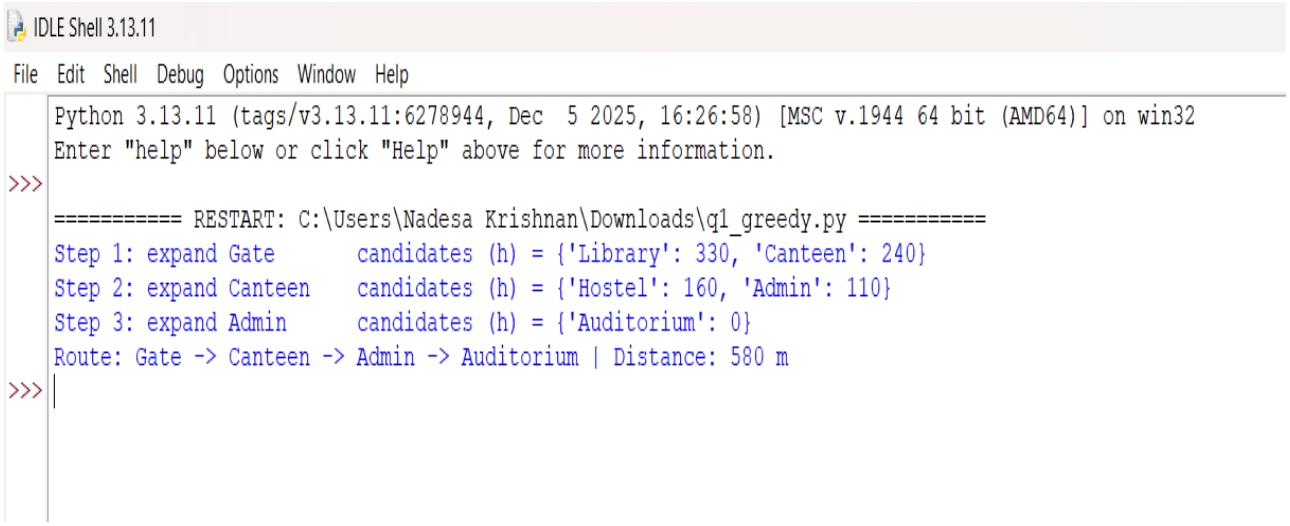
Program 1 (q1_greedy.py)

```
# Question 1 - Greedy Best-First Search (campus delivery robot)
roads = {
    "Gate":      {"Library": 120, "Canteen": 200},
    "Library":   {"Gate": 120, "Lab": 150, "Hostel": 300},
    "Canteen":   {"Gate": 200, "Hostel": 100, "Admin": 260},
    "Lab":       {"Library": 150, "Auditorium": 350},
    "Hostel":    {"Library": 300, "Canteen": 100, "Auditorium": 180},
    "Admin":     {"Canteen": 260, "Auditorium": 120},
    "Auditorium": {"Lab": 350, "Hostel": 180, "Admin": 120},
}
h = {"Gate": 400, "Library": 330, "Canteen": 240, "Lab": 280,
     "Hostel": 160, "Admin": 110, "Auditorium": 0}

def greedy(start, goal):
    current, visited, path, total = start, {start}, [start], 0
    step = 1
    while current != goal:
        cands = {n: h[n] for n in roads[current] if n not in visited}
        print(f"Step {step}: expand {current:<10} candidates (h) = {cands}")
        nxt = min(cands, key=cands.get)
        total += roads[current][nxt]
        visited.add(nxt); path.append(nxt); current = nxt; step += 1
    return path, total

if __name__ == "__main__":
    path, dist = greedy("Gate", "Auditorium")
    print("Route:", " -> ".join(path), "| Distance:", dist, "m")
```

Output :



Question 2 – A* Search

a) New strategy

The new rule f(n) = g(n) + h(n) is A* search.

- g(n) is the exact cost from the Gate to n; h(n) is the straight-line estimate from n to the Auditorium, so f(n) estimates the total cost of the best route passing through n.
- A* always expands the open node with the lowest f. It is guaranteed to return the shortest route provided h is admissible (never over-estimates the true remaining distance); with a consistent h it never needs to re-expand a node. A straight line can never be longer than a road, so this holds here (checked in part c).

b) Trace of the new rule

Step	Expanded	Candidate	g	h	f = g+h	Decision / open list
1	Gate (f=400)	Library	120	330	450	Open: Library 450, Canteen 440 Pick Canteen (lowest f)
		Canteen	200	240	440	
2	Canteen (f=440)	Hostel	300	160	460	Open: Library 450, Hostel 460, Admin 570 Pick Library
		Admin	460	110	570	
3	Library (f=450)	Lab	270	280	550	Hostel via Library: g=420, f=580 – worse than 460, discarded Open: Hostel 460, Lab 550, Admin 570 Pick Hostel
4	Hostel (f=460)	Auditorium	480	0	480	Open: Auditorium 480, Lab 550, Admin 570 Pick Auditorium
5	Auditorium (f=480)	–	480	0	480	Goal removed from open list with lowest f → stop

Route: Gate → Canteen → Hostel → Auditorium, distance = 200 + 100 + 180 = 480 m.

c) Comparison with Question 1

Location	Straight-line h	True remaining distance h*	h ≤ h* ?
Gate	400	480 (200+100+180)	Yes
Library	330	480 (300+180)	Yes
Canteen	240	280 (100+180)	Yes
Lab	280	350	Yes
Hostel	160	180	Yes
Admin	110	120	Yes

Every straight-line value is less than or equal to the true remaining road distance, so the heuristic is admissible and A* is guaranteed to give the shortest route. It is also consistent: h(n) ≤ cost(n, n') + h(n') for every road (for example Canteen→Hostel: 240 ≤ 100 + 160).

	Q1 – Greedy	**Q2 – A***
Route	Gate → Canteen → Admin → Auditorium	Gate → Canteen → Hostel → Auditorium
Distance	580 m	480 m
Nodes expanded	3 (Gate, Canteen, Admin)	4 (Gate, Canteen, Library, Hostel)
Optimal?	No	Yes

A* gives a route that is 100 m shorter (about 17% saving). It does a little more searching (one extra expansion) but guarantees the best answer, which is the trade-off the administration wanted.

Program 2 (q2_astar.py)

```
# Question 2 - A* Search: f(n) = g(n) + h(n)
import heapq
from q1_greedy import roads, h  # reuse the campus map from Q1

def a_star(start, goal):
    open_list = [(h[start], 0, start, [start])]
    best_g = {start: 0}
    step = 1
    while open_list:
        f, g, node, path = heapq.heappop(open_list)
        if g > best_g.get(node, float("inf")):
            continue
        print(f"Step {step}: expand {node:<10} g={g:<4} h={h[node]:<4} f={f}")
        step += 1
        if node == goal:
            return path, g
        for nb, w in roads[node].items():
            ng = g + w
            if ng < best_g.get(nb, float("inf")):
                best_g[nb] = ng
                print(f"        generate {nb:<10} g={ng:<4} h={h[nb]:<4} f={ng + h[nb]}")
                heapq.heappush(open_list, (ng + h[nb], ng, nb, path + [nb]))
    return None, float("inf")

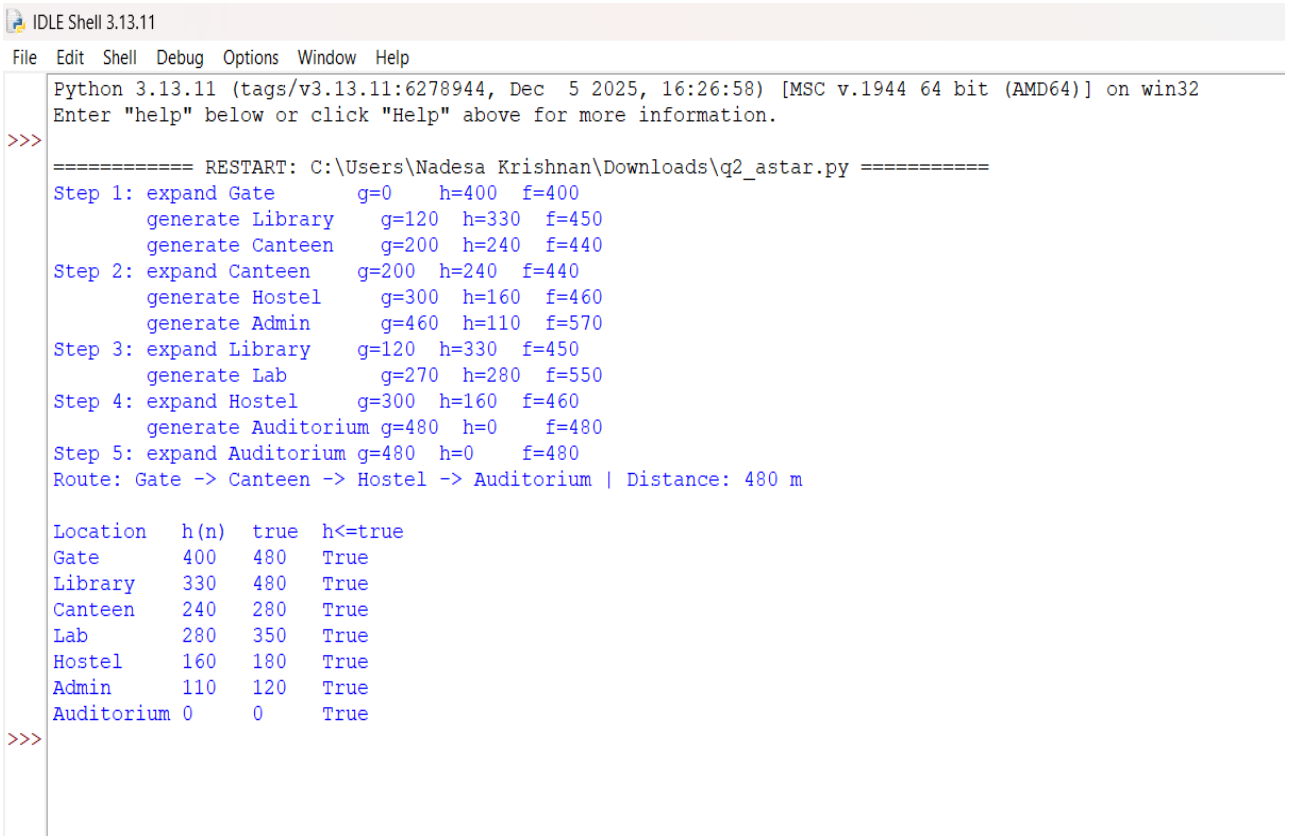
path, dist = a_star("Gate", "Auditorium")
print("Route:", " -> ".join(path), "| Distance:", dist, "m")

# Admissibility check: true remaining distance (Dijkstra) vs straight-line h
def true_dist(src, goal="Auditorium"):
    pq, seen = [(0, src)], {}
    while pq:
        d, n = heapq.heappop(pq)
        if n in seen: continue
        seen[n] = d
        for nb, w in roads[n].items():
```

```
            heapq.heappush(pq, (d + w, nb))
    return seen[goal]

print("\nLocation   h(n)  true  h<=true")
for n in roads:
    t = true_dist(n)
    print(f"{n:<10} {h[n]:<5} {t:<5} {h[n] <= t}")
```

Output :



Question 3 – Constraint Satisfaction Problem

a) Problem type and formulation

This is a Constraint Satisfaction Problem (CSP) – a scheduling problem equivalent to graph colouring (workshops = nodes, shared students = edges, time slots = colours).

- **Variables:** W1, W2, W3, W4, W5 (the slot of each workshop).
- **Domains:** {Morning, Afternoon, Evening} for each; after applying the unary constraints: D(W3) = {Afternoon, Evening}, D(W4) = {Morning, Afternoon}, D(W1) = D(W2) = D(W5) = all three.
- **Binary constraints (different slots):** W1≠W2, W1≠W3, W2≠W3, W2≠W4, W3≠W5, W4≠W5.
- **Unary constraints:** W3 ≠ Morning (trainer); W4 ≠ Evening (hall).

b) Building the timetable

Method: pick the variable with the fewest remaining slots (MRV); break ties by the most unscheduled conflicting workshops (degree), then by lower workshop number; try slots in the order Morning, Afternoon, Evening; after each assignment remove that slot from the unscheduled neighbours (forward checking).

Initial domains: W1 {M,A,E}, W2 {M,A,E}, W3 {A,E}, W4 {M,A}, W5 {M,A,E} (M = Morning, A = Afternoon, E = Evening)

Step	Workshop chosen and why	Assignment	Remaining slots of unscheduled workshops
1	W3 and W4 both have 2 slots; W3 has 3 unscheduled conflicts (W1, W2, W5) vs 2 for W4 → W3	W3 = Afternoon	W1 {M,E}, W2 {M,E}, W4 {M,A}, W5 {M,E}
2	All have 2 slots; unscheduled conflicts: W1=1, W2=2, W4=2, W5=1; tie W2/W4 → lower number W2	W2 = Morning	W1 {E}, W4 {A}, W5 {M,E}
3	W1 and W4 have 1 slot; unscheduled conflicts: W1=0, W4=1 → W4	W4 = Afternoon	W1 {E}, W5 {M,E}
4	W1 has 1 slot (fewest)	W1 = Evening	W5 {M,E}
5	Only W5 left; Morning tried first	W5 = Morning	–

Final valid timetable:

Slot	Workshops	Check
Morning	W2, W5	W2 and W5 share no students ✓
Afternoon	W3, W4	W3 and W4 share no students; W3 not in Morning ✓
Evening	W1	W4 not in Evening ✓

All six conflict pairs are in different slots (W1–W2: E/M, W1–W3: E/A, W2–W3: M/A, W2–W4: M/A, W3–W5: A/M, W4–W5: A/M) and both availability constraints hold.

c) Why MRV helps and what to do on a dead end ?

- **Why fewest-slots-first helps:** a variable with few legal values is the most likely to fail later. Scheduling it first ("fail-first") exposes dead ends early, near the root of the search tree, so large wrong subtrees are pruned instead of explored. Forward checking then shrinks the other domains immediately, and the degree tie-break picks the variable that constrains the most others.
- **If a workshop ends with no slot:** the current partial assignment cannot be completed, so backtrack – undo the latest assignment (restoring the removed slots), try the next slot for that workshop, and go further back if it also fails. If every option fails, the problem is unsolvable as stated and a constraint must be relaxed (e.g. add a fourth slot or free the trainer/hall).

Program 3 (q3_csp.py)

```
# Question 3 - CSP: workshop timetable with MRV + degree tie-break + forward checking
SLOTS = ["Morning", "Afternoon", "Evening"]
conflicts = [("W1","W2"),("W1","W3"),("W2","W3"),("W2","W4"),("W3","W5"),("W4","W5")]
domains = {f"W{i}": list(SLOTS) for i in range(1, 6)}
domains["W3"].remove("Morning")   # W3 trainer unavailable in the morning
domains["W4"].remove("Evening")   # W4 hall unavailable in the evening
neigh = {w: set() for w in domains}
for a, b in conflicts:
    neigh[a].add(b); neigh[b].add(a)

def pick(domains, assigned):
    un = [w for w in domains if w not in assigned]
    # fewest remaining slots -> most unscheduled conflicts -> lowest number
    return min(un, key=lambda w: (len(domains[w]),
               -len([n for n in neigh[w] if n not in assigned]), int(w[1:])))

def solve(domains, assigned):
    if len(assigned) == len(domains):
        return assigned
    w = pick(domains, assigned)
    for slot in domains[w]:                    # Morning, Afternoon, Evening
        new = {k: list(v) for k, v in domains.items()}
        new[w] = [slot]
        ok = True
        for n in neigh[w]:
            if n not in assigned:
                if slot in new[n]: new[n].remove(slot)
                if not new[n]: ok = False          # dead end -> backtrack
        if ok:
            a2 = dict(assigned); a2[w] = slot
            rem = {k: v for k, v in new.items() if k not in a2}
            print(f"Assign {w} = {slot:<9} remaining domains: {rem}")
            res = solve(new, a2)
            if res: return res
```

```
    return None

print("Initial domains:", domains)
sol = solve(domains, {})
print("\nTimetable:")
for w in sorted(sol): print(f"  {w}: {sol[w]}")
assert all(sol[a] != sol[b] for a, b in conflicts)
assert sol["W3"] != "Morning" and sol["W4"] != "Evening"
print("All constraints satisfied.")
```

Output :



Question 4 – Minimax

a) Technique and assumption

The bot should use the Minimax algorithm (two-player, zero-sum, perfect information). X is the MAX player and O is the MIN player. The assumption is that Player O plays optimally (rationally) and always picks the reply that minimises X's score, so X chooses the move whose worst-case outcome is best.

b) Values of all nodes

Tree: Root (X moves, MAX) → two first moves A, B → O replies (MIN) → X's final moves (MAX) → leaf scores.

Level / node	Type	Children values	Node value
After A, reply 1 (leaves 4, 7)	MAX (X)	4, 7	7
After A, reply 2 (leaves 8, 2)	MAX (X)	8, 2	8
After B, reply 1 (leaves 3, 5)	MAX (X)	3, 5	5
After B, reply 2 (leaves 6, 9)	MAX (X)	6, 9	9
Move A (O to reply)	MIN (O)	7, 8	7
Move B (O to reply)	MIN (O)	5, 9	5
Root – starting position	MAX (X)	7, 5	7

c) Best move, guaranteed score and play

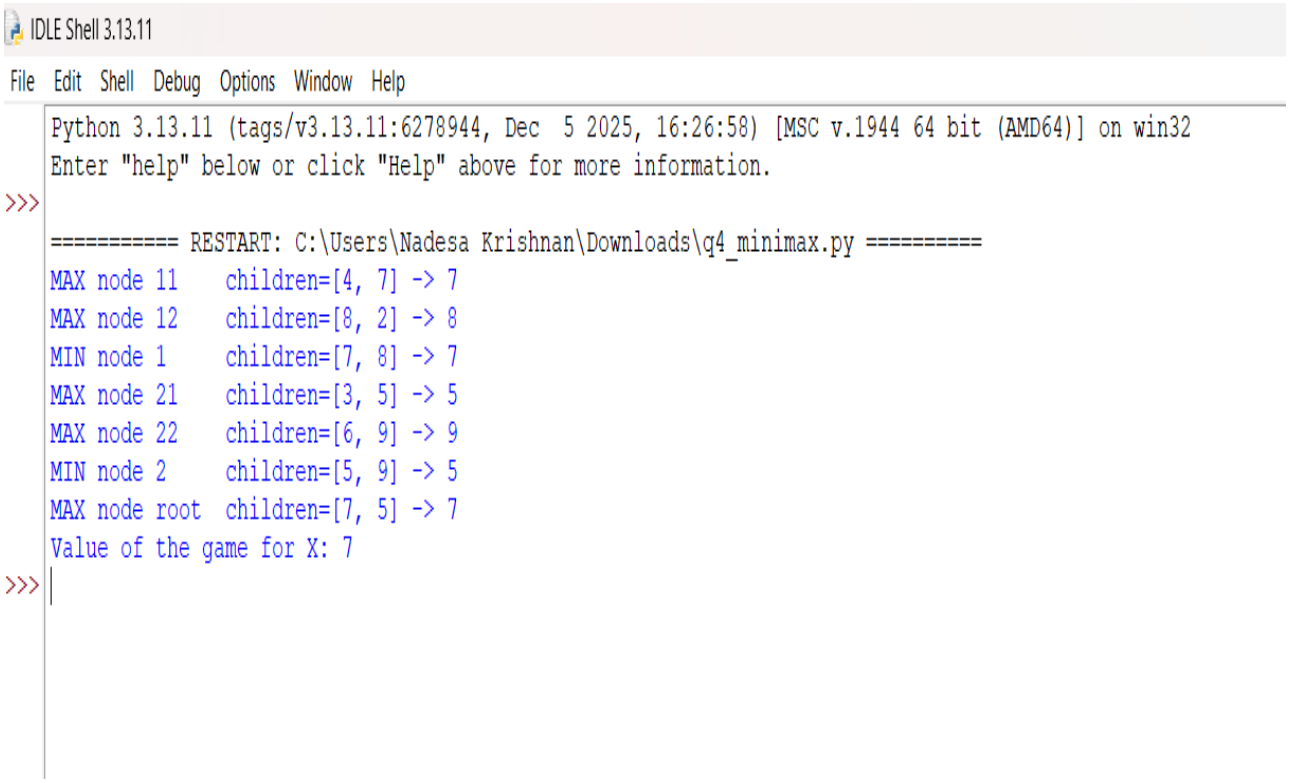
- Best first move: the left move (A), since 7 > 5.
- Guaranteed score: 7 – X can secure at least 7 whatever O does.
- Best play: X plays the left first move → O replies with the left reply (leaving X a maximum of 7, rather than 8 after the right reply) → X plays its second (right) final move, scoring 7. Choosing the right first move would let O hold X to 5, so it is worse.

Program 4 (q4_minimax.py)

```
# Question 4 - Minimax on a Max-Min-Max tree
tree = [  # X move -> O reply -> X final-move scores
    [[4, 7], [8, 2]],   # first move A (left)
    [[3, 5], [6, 9]],   # first move B (right)
]
def minimax(node, is_max, label):
    if isinstance(node, int):
        return node
    vals = [minimax(c, not is_max, f"{label}{i+1}") for i, c in enumerate(node)]
    v = max(vals) if is_max else min(vals)
    print(f"{'MAX' if is_max else 'MIN'} node {label or 'root':<5} children={vals} -> {v}")
    return v

best = minimax(tree, True, "")
print("Value of the game for X:", best)
```

Output :


IDLE Shell 3.13.11
File Edit Shell Debug Options Window Help
Python 3.13.11 (tags/v3.13.11:6278944, Dec 5 2025, 16:26:58) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:\Users\Nadesa Krishnan\Downloads\q4_minimax.py ==========
MAX node 11 children=[4, 7] -> 7
MAX node 12 children=[8, 2] -> 8
MIN node 1 children=[7, 8] -> 7
MAX node 21 children=[3, 5] -> 5
MAX node 22 children=[6, 9] -> 9
MIN node 2 children=[5, 9] -> 5
MAX node root children=[7, 5] -> 7
Value of the game for X: 7
>>>

Question 5 – Alpha-Beta Pruning

a) Evaluation left to right with pruning

The engine is MAX (alpha = best value guaranteed for it so far); the opponent's replies are MIN nodes (beta = best value for the opponent so far). A branch is cut when beta ≤ alpha.

Move	Position scored	alpha	beta	Action
M1	6	$-\infty$	6	continue
M1	9	$-\infty$	6	min stays 6
M1	7	$-\infty$	6	M1 = 6 → alpha = 6
M2	4	6	4	beta 4 ≤ alpha 6 → PRUNE; skip 8 and 10 (M2 ≤ 4)
M3	7	6	7	continue
M3	5	6	5	beta 5 ≤ alpha 6 → PRUNE; skip 12 (M3 ≤ 5)

Skipped positions: 8 and 10 under M2, and 12 under M3.

b) Result

- Positions actually scored: 6 of 9 (3 for M1, 1 for M2, 2 for M3).
- Move chosen: M1, with value 6.
- Check against full minimax: min(M1) = 6, min(M2) = 4, min(M3) = 5, and max(6, 4, 5) = 6 → move M1, value 6. The result is identical; pruning only removed positions that could not change the decision.

c) Move ordering and the evaluation function

- Effect of ordering: pruning is strongest when the best moves are examined first, because a high alpha is established early and more later branches are cut. With the order M2, M1, M3, 8 positions would be scored (only 12 skipped). If, in addition, the refuting reply is examined first in each opponent node (e.g. M3 as 5, 7, 12), only 5 positions are scored. In the worst ordering nothing is pruned and all 9 are scored. The final answer never changes; only the work does. With perfect ordering alpha-beta can search about twice as deep as plain minimax in the same time.
- Evaluation function is needed: the game tree is too large to search to terminal positions, so the search stops at a fixed depth. The evaluation function gives a numeric estimate of how good a non-terminal position is for the engine (using features such as material, mobility or position). Without it the engine would have no values to back up, and the quality of its play depends directly on how good this estimate is.

Program 5 (q5_alphabeta.py)

```
# Question 5 - Alpha-Beta pruning (Max root, Min replies)
moves = {"M1": [6, 9, 7], "M2": [4, 8, 10], "M3": [7, 5, 12]}
alpha, beta = float("-inf"), float("inf")
best_move, scored = None, 0
for m, replies in moves.items():
    b = beta
    val = float("inf")
    for i, s in enumerate(replies):
        scored += 1
        val = min(val, s); b = min(b, val)
        print(f"{m}: score {s}  -> best so far for MIN (beta) = {b}, alpha = {alpha}")
        if b <= alpha:
            skipped = replies[i+1:]
            print(f"   PRUNE {m}: beta {b} <= alpha {alpha}; skipped {skipped}")
            break
    if val > alpha:
        alpha, best_move = val, m
    print(f"   {m} value = {val if val>=alpha else '<= ' + str(val)} | alpha = {alpha}")
print(f"\nPositions scored: {scored} of 9 | Best move: {best_move} | Value: {alpha}")
full = max(min(r) for r in moves.values())
print("Full minimax value:", full, "-> same result:", full == alpha)
```

Output :

